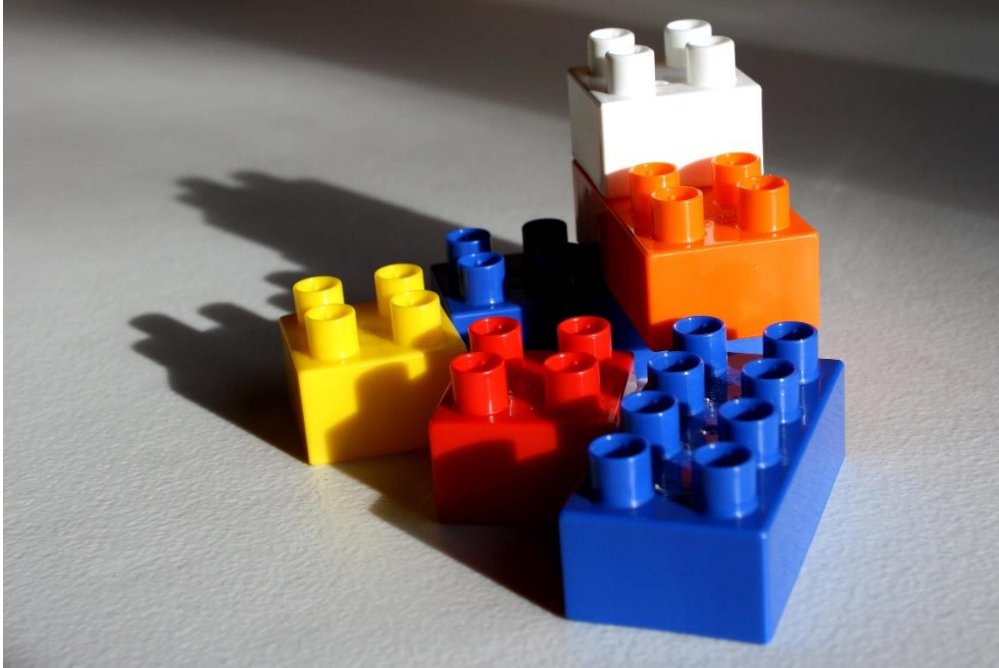


LEGO

DATA SCIENCE REPORT



by Kadir Arslan

Created: 01.12.2024

Last Edited: 10.03.2026

LEGO Report

In this report, we will examine the data sets related to the LEGO Company.

These data sets will help us understand the company's history, its product range, and its distinctive features.

Using data science methods, we will explore and address a range of questions, including the following:

- What is the most enormous LEGO set ever created and how many parts did it have?
- How many different colors were used on different LEGO sets?
- In which year were the first LEGO sets released and how many sets did the company sell when it first launched?
- Which LEGO theme has the most sets? Is it Harry Potter, Ninjago, Friends or something else?
- When did the LEGO company really take-off based on its product offering? How many themes and sets did it release every year?
- Did LEGO sets grow in size and complexity over time? Do older LEGO sets tend to have more or fewer parts than newer sets?
- And more...

The data is provided in a Comma-Separated Values (CSV) file.

All data sets used in this project:

colors.csv: [github.com](#)

sets.csv: [github.com](#)

themes.csv: [github.com](#)

We will be using python and its libraries *pandas*, *seaborn*, *matplotlib*:

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Converting the Raw CSV Files into Data frames

We are going to start with the dataset “colors”:

```
# Colors that are used in Lego sets:
colors = pd.read_csv('csv_files/colors.csv')
```

The dataset “colors” shows the colors used in Lego products through the years.

Determining of the Shape of a Dataset

To get the number of columns and rows in a dataset, we use the **.shape()** function:

```
print(colors.shape)

>> (135, 4)
```

The analysis shows that there are 135 rows and 4 columns in this dataset.

Obtaining and Examining Initial Insight into the Dataset:

To get first rows of a dataset we use **.head(n)** function from pandas. (Default n=5)

```
print(colors.head())
```

This gives an insight into the first five rows of the dataset:

id	name	rgb	is_trans
-1	Unknown	0033B2	f
0	Black	05131D	f
1	Blue	0055BF	f
2	Green	237841	f
3	Dark Turquoise	008F9B	f

The findings demonstrate that there are 5 columns, which include the name of the color and its corresponding RGB values.

Determining the Number of Unique Values in a Dataset

To find the number of unique colors, all we need to do is to check if every entry in the name column is unique:

```
print(colors['name'].nunique())
```

```
>> 135
```

The results indicate that a total of 135 unique colors were deployed in LEGO sets.

Counting the Occurrence of Values in a Dataset

To find the number of transparent colors, we use `.value_counts()` function.

```
print(colors.is_trans.value_counts())
```

	is_trans
f	107
t	28

We see that there are 28 transparent colors and 107 not transparent colors used in Lego sets

Lego Sets

The file sets.csv contains information on LEGO sets, specifying the years of releases and the number of parts for each set.

Obtaining and Examining Initial Insight into the Dataset:

```
sets = pd.read_csv('csv_files/lego_files/sets.csv')
print(sets.head())
```

	set_num		name	year	theme_id	num_parts
0	001-1		Gears	1965	1	43
1	0011-2	Town	Mini-Figures	1978	84	12
2	0011-3	Castle 2 for 1 Bonus Offer		1987	199	0
3	0012-1	Space	Mini-Figures	1979	143	12
4	0013-1	Space	Mini-Figures	1979	143	12

Sorting a Dataset According to the Values of a Column

The dataset is sorted by the number of parts to determine which LEGO sets contain the most components.

```
sets = sets.sort_values('num_parts', ascending=False)
print(sets.head())
```

	set_num		name	year	theme_id	num_parts
15004	BIGBOX-1	The Ultimate Battle for Chima		2015	571	9987
11183	75192-1	UCS Millennium Falcon		2017	171	7541
10551	71043-1	Hogwarts Castle		2018	246	6020
295	10256-1	Taj Mahal		2017	673	5923
221	10189-1	Taj Mahal		2008	673	5922



It becomes apparent that the set *“The Ultimate Battle for Chima”*, released in 2015, had the greatest number of parts with 9987 parts.

Sorting by the Column ‘Year’.

To determine the release year of the first LEGO set, we will sort the sets in ascending order by release year.

```
sets = sets.sort_values('year', ascending=True)
print(sets.head())
```

	set_num		name	year	theme_id	num_parts
9544	700.A-1		Small Brick Set (ABB)	1949	371	24
9539	700.3-1		Medium Gift Set (ABB)	1949	365	142
9545	700.B-1	Small Doors and Windows Set (ABB)		1949	371	12
9534	700.2-1		Large Gift Set (ABB)	1949	365	178
9521	700.1-1		Extra-Large Gift Set (ABB)	1949	365	142

The outcomes establish that the Lego has first started releasing its products in **1949**.

Filtering the Dataset by Applying a Condition to a Column

Now we would like to determine the number of sets released in 1949.

```
sets_from_1945 = sets[sets['year'] == 1949]
print("\nNumber of sets from the year 1949:")
print(sets_from_1945.value_counts().sum())
```

```
>> Number of sets from the year 1949:
```

```
>> 5
```

The inspection highlights that LEGO has started its journey with 5 different sets in 1949.

Visualization of the number of Sets Published over Time

Now let's take a look at how many sets the LEGO company has published year-on-year.

This might tell us something about how LEGO's product offering has changed over time.

We are going to create a new Series called **“sets_by_year”** which has the years as the index and the number of sets as the value.

The CSV file dates from late 2020; the data for the last two years is incomplete. To ensure that the entire calendar years are represented, we will exclude the data for those two years before creating the graph.

Sorting the Sets by year:

```
sorted_sets_by_year = sets.sort_values('year')
sorted_sets_by_year = sorted_sets_by_year.year.value_counts()
print(sorted_sets_by_year.head(7))
```

Output:

```
year
2019    840
2018    816
2017    786
2015    774
2016    757
2014    690
2020    674
Name: count, dtype: int64
```

Grouping a Dataset by a Certain Column

The set will be grouped into years to be used in further corresponding tasks.

```
sets_by_year = sets.groupby('year')
```

This function does not perform any operation by itself and serves as a boiler point for implementing additional functions. For example, we are going to apply **.agg()** function on this set.

Sorting the Dataset by year:

This will be first step to obtain the desired dataset.

We are going to use **.count()** function to build the graph on the data. This function is different than **.value_counts()** function which concentrates just to get value-count graph.

```
sets_by_year_counts = sets_by_year.count()
print(sets_by_year_counts["set_num"].head())
```

year	
1949	5
1950	6
1953	4
1954	14
1955	28

Now we have a dataset which displays the yearly distribution of released LEGO sets.

Excluding Certain Entries using Python Slicing Techniques

Here, we don't want to have last two years' data since they were incomplete.

```
sets_by_year_counts = sets_by_year_counts[:-2]
```


Creating a Line Chart (Seaborn)

Visualization of number of Sets over Year:

We have transformed the raw data into a data set grouped by year, where each group contains the properties corresponding to that year.

Next, we create a line plot to visualize the number of LEGO sets released per year.

For this purpose, we use the Seaborn library to generate the line chart.

Code:

```
sns.set_theme(style="whitegrid", context="talk")

fig, ax = plt.subplots(figsize=(12,6))

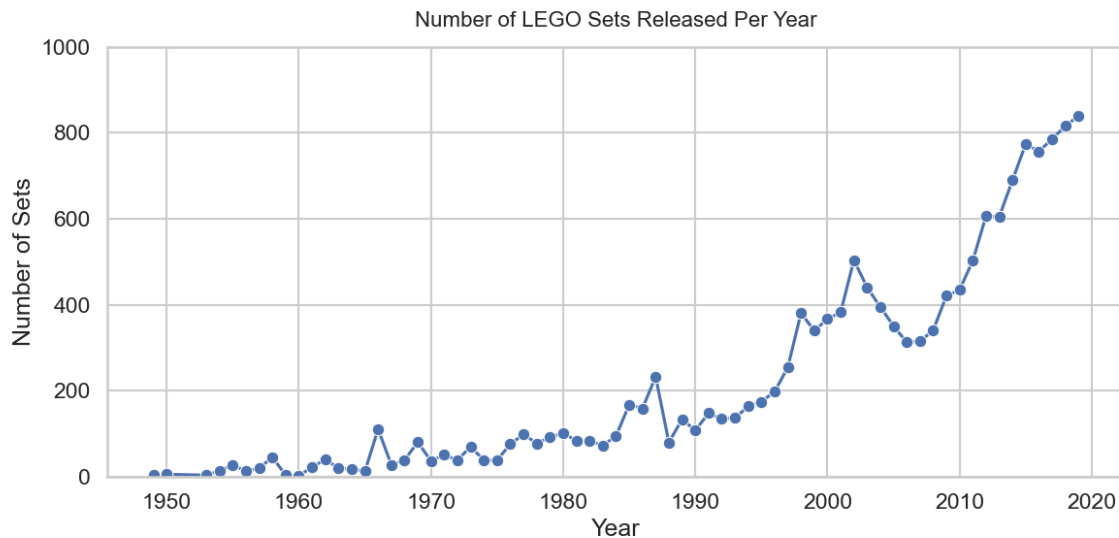
sns.lineplot(
    data=sets_by_year_counts,
    x=sets_by_year_counts.index,
    y="set_num",
    marker="o"
)

ax.set_title("Number of LEGO Sets Released Per Year", fontsize=16, pad=15)
ax.set_xlabel("Year")
ax.set_ylabel("Number of Sets")

ax.set_ylim(0, 1000)

plt.tight_layout()
plt.show()
```

Line Plot:



This Figure demonstrates that:

From 1949 until the mid-1980s, LEGO experienced steady and moderate growth.

In 1988, LEGO experienced a dramatic drop of approx. 60% in the number of sets.

- The reason for this may be that they lost the sole patent rights to their products that year.

Between 1990-2002, there is a significant expansion of the product range.

- Fearing the rise of video games and new competitors, LEGO aggressively and unsustainably expanded its product lineup.

Between 2002-2007 there is a sharp decrease in the number of LEGO sets.

- The company was on the brink of bankruptcy due to a bloated supply chain. In response, the new management launched a comprehensive restructuring strategy and reduced the number of parts.

Since 2008, the company has been experiencing significant and increasing growth:

- By stabilizing its core production operations and increasing efficiency, LEGO has entered a period of explosive and planned growth.

Comparison of number of sets and number of unique themes by year

Now we want to determine the relationship between the number of sets and number of different themes throughout the years.

Statistical Aggregation of a Grouped Data

The set `sets_by_year` was the set which was grouped on “year”.

To obtain the number of different themes on year, we use `.agg()` function on the set “`sets_by_year`”. This set was the set which was grouped on “year”.

```
themes_by_year = sets_by_year.agg({'theme_id': pd.Series.nunique})
```

- After grouping the dataset by year and counting the number of unique theme IDs, it now consists of two columns: “Years” and the number of unique (*pd.Series.unique*) themes in that year.

Again, excluding last two years of data:

```
themes_by_year = themes_by_year[:-2]
```

Changing the Name of a Column of a Dataset

Now that we have the number of different theme IDs instead of direct IDs of themes in the second column, we should rename the second column.

```
themes_by_year.rename(columns = {'theme_id': 'nr_themes'}, inplace = True)
```

It is always better to rename the column after using grouping and aggregating (*.agg()*) functions since that performs a significant operation.

Now, the dataset is ready for this task.

```
print(themes_by_year.head())
```

	nr_themes
year	
1949	2
1950	1
1953	2
1954	2
1955	4

Creating Dual-Axis Line Plot (matplotlib and seaborn)

We had a dataset which shows the number of published sets by year.

Now we have a second dataset which shows the number of unique sets by year.

We can compare the number of sets and number of themes by year using these two datasets on the same line graph.

We will have two different y-axes showing the number of sets and topics, and an x-axis showing the years.

Code:

```
sns.set_theme(style="white", context="talk")

# Figure size and primary axis
fig, ax1 = plt.subplots(figsize=(14, 7))

# Chart title
plt.title('Evolution of Sets and Themes Over Time',
          fontsize=18, fontweight='bold', pad=20)

# colors
color_sets = "#2a9d8f"
color_themes = "#e76f51"

# first line on ax1
```

```

ax1.plot(sets_by_year_counts.index,
        sets_by_year_counts.set_num,
        color=color_sets, linewidth=3, label='Number of Sets')

# ax1 Labels and ticks
ax1.set_xlabel('Year', fontsize=14, fontweight='bold', labelpad=15)
ax1.set_ylabel('Number of Sets', color=color_sets, fontsize=14, fontweight='bold')
ax1.tick_params(axis='y', labelcolor=color_sets)

# Adding subtle gridlines to ax1
ax1.grid(True, linestyle='--', alpha=0.6)

# Creating second axis
ax2 = ax1.twinx()

# Plotting the second line
ax2.plot(themes_by_year.index,
        themes_by_year.nr_themes,
        color=color_themes, linewidth=3, label='Number of Themes')

# ax2 Labels and ticks
ax2.set_ylabel('Number of Themes', color=color_themes, fontsize=14,
fontweight='bold')
ax2.tick_params(axis='y', labelcolor=color_themes)

# Plotting the Legend
lines_1, labels_1 = ax1.get_legend_handles_labels()
lines_2, labels_2 = ax2.get_legend_handles_labels()
ax1.legend(lines_1 + lines_2, labels_1 + labels_2, loc='upper left', frameon=True,
shadow=True)

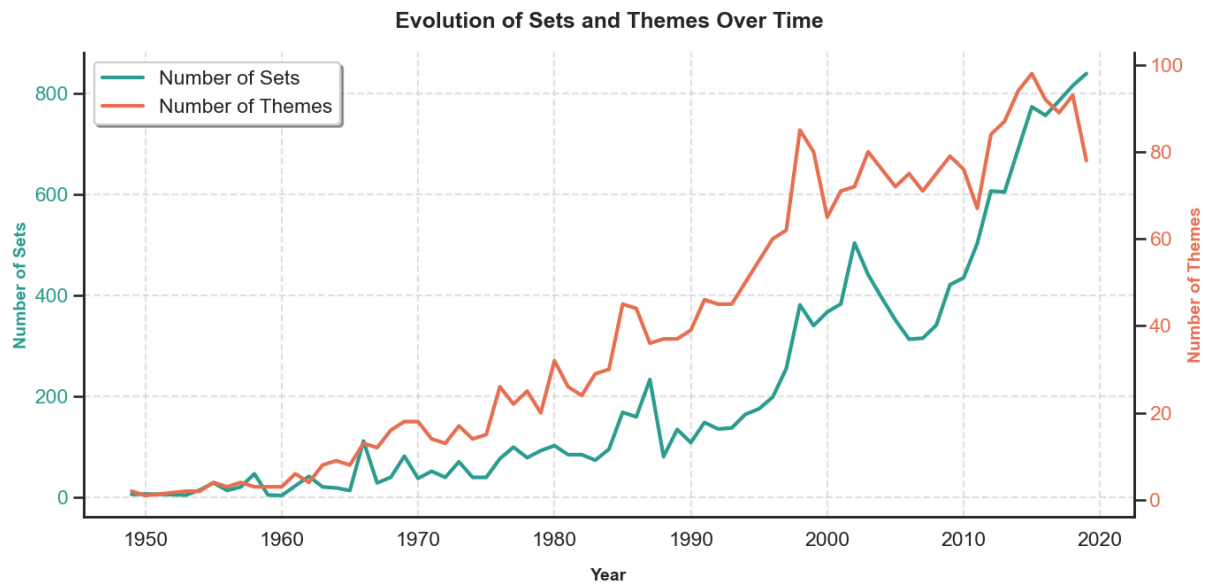
# Clean up the top border (despine) while keeping the right axis
sns.despine(top=True, right=False)

# Adjust layout so nothing gets cut off
fig.tight_layout()

# Display the chart
plt.show()

```

Dual-Axis Line Plot:



The chart shows that the number of themes and sets released and sold by LEGO grew together consistently well into the early 2000s.

After this period, it becomes apparent that Lego has increased its number of sets while keeping the number of themes in a steady position.

These results are consistent with previous findings which were obtained on the first graph.

Average Number of Sets over Years

Now we want to examine average numbers of parts over time.
We want to see how it has changed throughout the years.

Statistical Aggregation of a Grouped Data

First, we want to get an appropriate dataset for this:

The set “sets_by_year” was the set which was grouped on “year”.

Now we are applying `.agg()` function with getting average values on number of parts of sets (`pd.Series.mean`) each year.

```
parts_per_set = sets_by_year.agg({'num_parts': pd.Series.mean})
```

As before we want to exclude last two years of data (they were not completed)

```
parts_per_set = parts_per_set[:-2]
```

As before, we would like to rename the column after aggregate operation.

```
parts_per_set.rename(columns = {'num_parts': 'avg_num_parts' }, inplace = True)
```

Creating Scatter Graph (matplotlib and seaborn)

Code:

```
sns.set_theme(style="whitegrid", context="talk")

plt.figure(figsize=(12, 6))

# Plot using Seaborn's scatterplot

# 's' makes the dots larger
# 'alpha' adds slight transparency so overlapping dots are visible
# 'edgecolor' adds a crisp border to each dot
sns.scatterplot(
```

```

x=parts_per_set.index,
y=parts_per_set.avg_num_parts,
color="#219ebc",
s=120,          # Marker size
alpha=0.8,      # Transparency
edgecolor="black", # Crisp border around markers
linewidth=0.5
)

# Titles and axis labels
plt.title('Average Number of Parts per Set Over Time', fontsize=18,
fontweight='bold', pad=20)
plt.xlabel('Year', fontsize=14, fontweight='bold', labelpad=10)
plt.ylabel('Average Number of Parts', fontsize=14, fontweight='bold', labelpad=10)

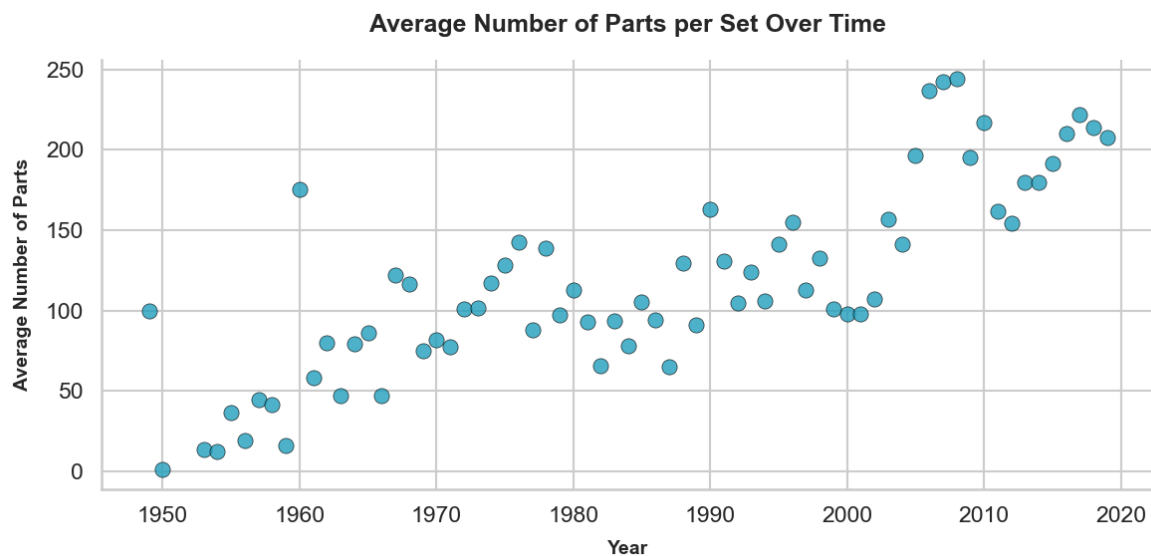
# Remove the top and right borders for a cleaner look
sns.despine(top=True, right=True)

# Adjust layout to prevent clipping
plt.tight_layout()

# Display the chart
plt.show()

```

Scatter graph:



The scatter graph indicates that the average amount of individual parts (bricks or other components) in Lego sets has steadily increased over time.

That makes perfect sense, because as LEGO's customer base grows, more sophisticated and comprehensive games would be needed.

Number of sets per Lego theme:

There are many different themes in LEGO. (i.e. Airport, Building, Cargo, Star Wars etc.)

Here, we want to examine which Lego sets are used more frequently and how many times these sets are released.

Recalling “sets” data:

```
sets = pd.read_csv('csv_files/lego_files/sets.csv')
print(sets.head())
```

	set_num		name	year	theme_id	num_parts
0	001-1		Gears	1965	1	43
1	0011-2	Town	Mini-Figures	1978	84	12
2	0011-3	Castle 2 for 1 Bonus Offer		1987	199	0
3	0012-1	Space	Mini-Figures	1979	143	12
4	0013-1	Space	Mini-Figures	1979	143	12

Now, we would like to get a dataset which shows the most used themes in LEGO.

The easiest way of doing this is using `.value_counts()`:

```
set_theme_count = sets["theme_id"].value_counts()
```

`value_counts()` function returns a dataset with indexes and values, so we are going to convert it into a data frame with two distinguished columns:

```
set_theme_count = pd.DataFrame({'id': set_theme_count.index, 'set_count':  
set_theme_count.values})
```

Themes IDs refer to the same theme, so we create a new set from the first set to get the number of counts of these Lego sets.

Getting an insight:

```
print(set_theme_count.head())
```

	id	set_count
0	158	753
1	501	656
2	494	398
3	435	356
4	503	329

We use IDs here because we have also another dataframe which uses the same IDs of the Lego themes and it has the names of these themes according to IDs.

Here we have another csv file.

```
themes = pd.read_csv('csv_files/lego_files/themes.csv')
```

Getting the Initial Insight:

```
print(themes.head())
```

	id	name	parent_id
0	1	Technic	NaN
1	2	Arctic Technic	1.0
2	3	Competition	1.0
3	4	Expert Builder	1.0
4	5	Model	1.0

The findings show that the dataset has 3 columns. It has name column which shows the names of the themes. It distinguishes the themes by specifying the IDs. Lastly, it has also a parent ID section which determines if the theme is subjected to another theme.

Merging two Different Datasets on one Common Feature

We are going to merge these two data frames to receive IDs, names and the amount of sets of these themes published by LEGO on the same dataframe.

Here, we will utilize panda's `.merge()` function.

This function takes the names of the sets as input, along with the "on=" section, which specifies the column in which the set should be merged.

```
merged_df = pd.merge(set_theme_count, themes, on='id')
```

Sorting the Merged Dataset and Getting the first Glimpse:

```
merged_df.sort_values('set_count', ascending=False)

print("\n Merged Dataframe")
print(merged_df.head())
```

Merged Dataframe

	id	set_count	name	parent_id
0	158	753	Star Wars	NaN
1	501	656	Gear	NaN
2	494	398	Friends	NaN
3	435	356	Ninjago	NaN
4	503	329	Key Chain	501.0

Now we would like to create a bar chart showing the 10 most frequently used themes in LEGO along with the total number of publications for each:

Visualization Using a Bar Chart

Code:

```
sns.set_theme(style="whitegrid", context="talk")

fig, ax = plt.subplots(figsize=(14, 7))
```

```

# Create the bar plot

# 'hue', distinguishes the colors of the components.
# 'palette' specifies the color gradient.
# We are going to use palette "muted" here. (it is from seaborn)
# 'edgecolor' adds a subtle outline to the bars.
# We set legend=False since the x-axis already tells us the names

sns.barplot(
    x=merged_df.name[:10],
    y=merged_df.set_count[:10],
    hue=merged_df.name[:10],
    palette="muted",
    edgecolor=".2",
    width=0.8,
    legend=False
)

plt.title('Top 10 Themes by Number of Sets', fontsize=18, fontweight='bold',
pad=20)
plt.xlabel('Theme Name', fontsize=14, fontweight='bold', labelpad=10)
plt.ylabel('Number of Sets', fontsize=14, fontweight='bold', labelpad=10)

# Format the tick marks
# It is a precaution for the long names at display.
# Wrapping the labels
labels = ax.get_xticklabels()
new_labels = [textwrap.fill(label.get_text(), 12) for label in labels]

# Reapply them
ax.set_xticks(ax.get_xticks())
ax.set_xticklabels(new_labels)

plt.xticks(fontsize=12, fontweight='bold')
plt.yticks(fontsize=12)

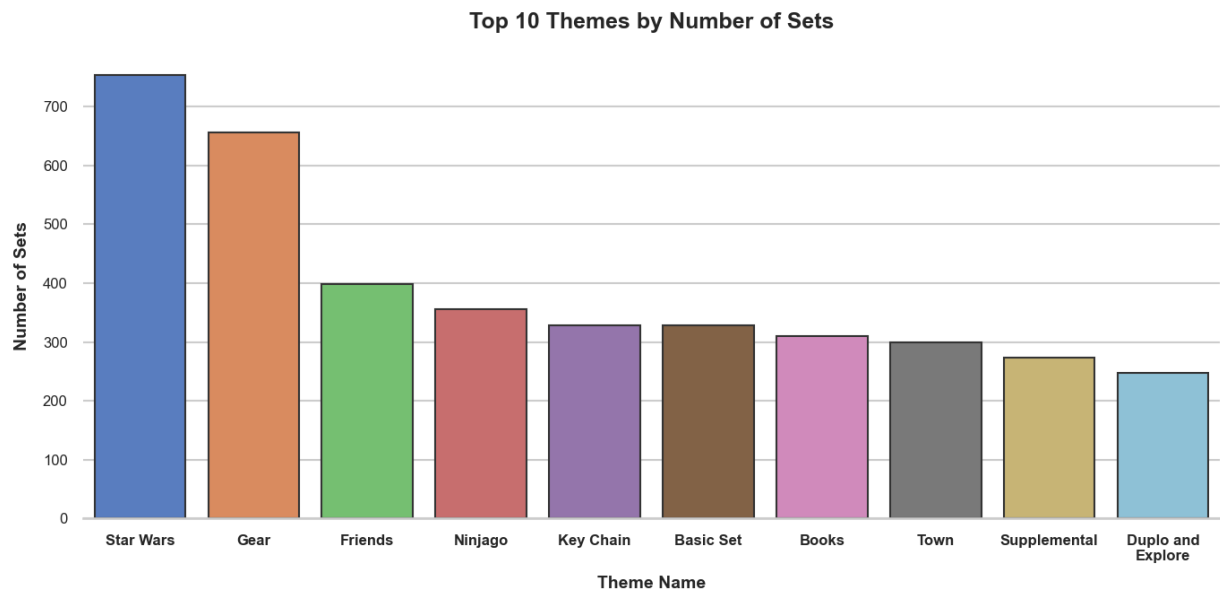
# Clean up the borders (despine)
# Removing some borders to design the chart lighter and more modern.
sns.despine(left=True, top=True, right=True)

# Adjust layout
plt.tight_layout()

# Display the chart
plt.show()

```

Graph:



The Figure illustrates that Star Wars is by far most used Lego theme in Lego History. Lovers of these Lego Themes had more than 700 different options to enjoy passion.

The beloved theme “Friend” ranks third with 400 sets on Lego history.



Conclusion

In this report, we have examined various key data science methods used for data analysis, transformation, and visualization. We applied the `.value_counts()` method to examine the distributions within the datasets, and `groupby()` together with `count()` to aggregate information across different categories. We also utilized the `.agg()` function to perform operations on specific columns and applied `rename()` to improve the clarity and readability of DataFrame structures. Data slicing with square-bracket notation allowed us to efficiently select subsets of data for further analysis.

In addition, we created both single-axis and dual-axis line charts using Matplotlib and Seaborn, enabling the visualization of variables with different scales in the same figure. Scatter plots and bar charts were generated to better understand relationships and comparisons within the data. We also worked with relational database concepts by using primary and foreign keys, and combined datasets effectively by applying the `.merge()` function on shared columns.

Overall, these methods illustrate a modern data science workflow that integrates data processing, aggregation, database logic, and visualization to derive meaningful insights from structured datasets.

References

<https://www.udemy.com/course/100-days-of-code/>

https://pandas.pydata.org/docs/user_guide/index.html#user-guide

<https://matplotlib.org/stable/users/index.html>

<https://seaborn.pydata.org/tutorial.html>

<https://trends.google.com/home>

<https://euobserver.com/104064/eu-court-rules-lego-bricks-can-be-made-by-rivals/#:~:text=Lego%20had%20won%20an%20EU,Lego's%20trademark%20on%20its%20bricks.>

<https://www.kitstore.de/p/lego-r-star-warstm-75402-arc-170-starfightertmm>

<https://www.osiander.de/shop/home/artikeldetails/A1072987707>